Компиляторы. Распределение регистров

Косарев Дмитрий

19 и 26 октября 2024 г.

Дата сборки: 19 сентября 2025 г.



В этих слайдах

Краткий разбор алгоритмов распределения регистров

1. Раскраска графов

Оглавление

1. Раскраска графов

Оглавление

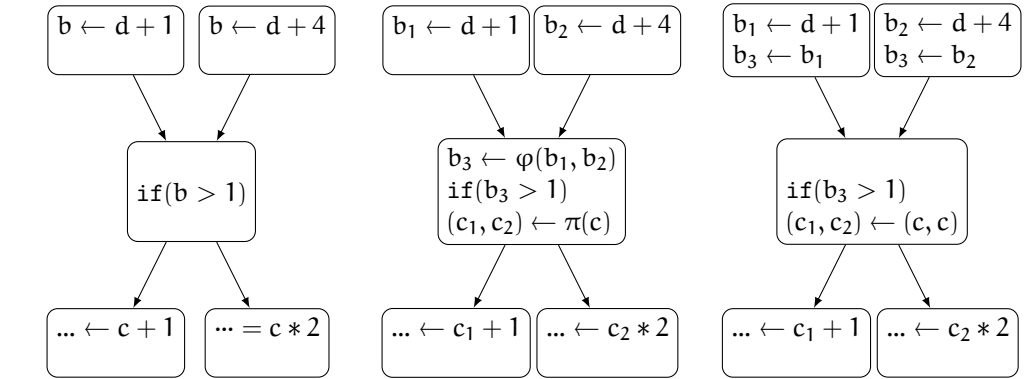
1. Раскраска графов

Оглавление

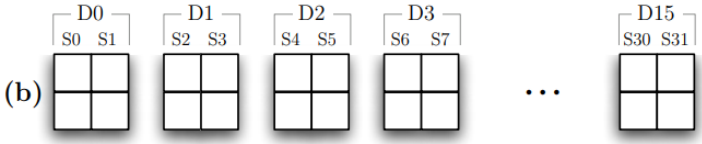
1. Раскраска графов

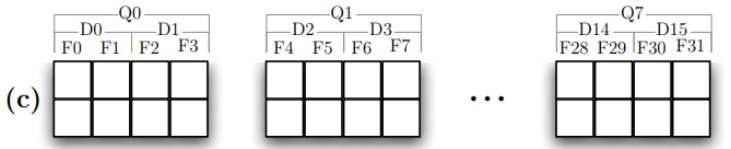
Подготовка

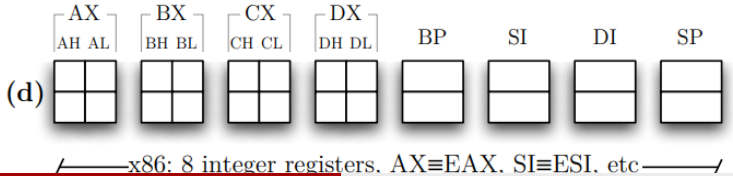
За счет этого укорачиваются появляются новые переменные с укороченным live range. Также перед всем эти стоит сделать протягивания констант и dead code elimination, чтобы избавиться от ненужных φ.



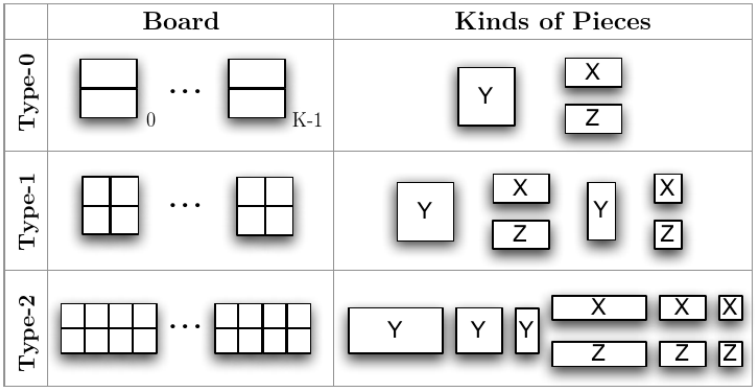
Подробнее гуглить про Static Single Information (SSI) форму


(b)
D0
S0 S1
D1
S2 S3
D2
S4 S5
D3
S6 S7
D15
S30 S31
ARM: 16 double precision floating point registers


(c)
Q0
D0
D1
F0 F1 F2 F3
Q1
D2
D3
F4 F5 F6 F7
Q7
D14
D15
F28 F29 F30 F31
SPARC V8: 8 quad-precision floating point registers


(d)
AX
AH AL
BX
BH BL
CX
CH CL
DX
DH DL
BP
SI
DI
SP
x86: 8 integer registers, AX≡EAX, SI≡ESI, etc

Виды пазлов



- Тип 0: PowerPC & ARM integers
- Тип 1: ARM float
- Тип 2: SPARC V8
- Гибрид 0+1: x86
- Гибрид 1+2: SPARC V9

Картинки из статьи «Register allocation by puzzle solving» (2008) [3]

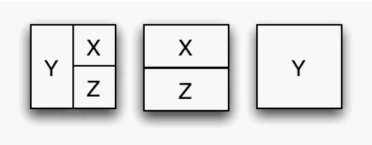
Абстракция пазлов

- Пазл = доска (зоны = регистры) + фигурки (переменные)

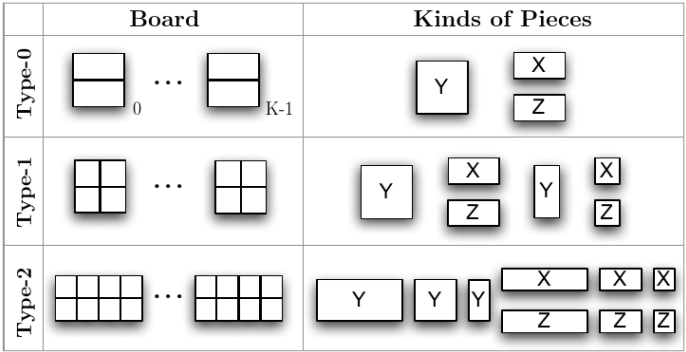




- Фигурки не могут перекрываться
- Некоторые фигурки могут быть уже выложены на доску
- Задача: уместить оставшиеся фигурки на доску (**register allocation**)



Создание фигурок пазла



Для каждой инструкции в программе i:

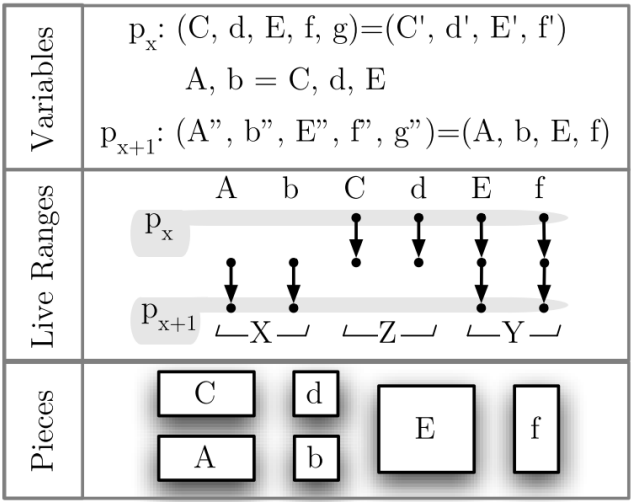
- Создать по одной фигурке для live-in и live-out переменных
- Если переменная перестает быть живой ниже i, тогда тип фигурки X
- Если переменная начинает быть живой в i, тогда Z
- Иначе Y

Пример. Создание фигурок

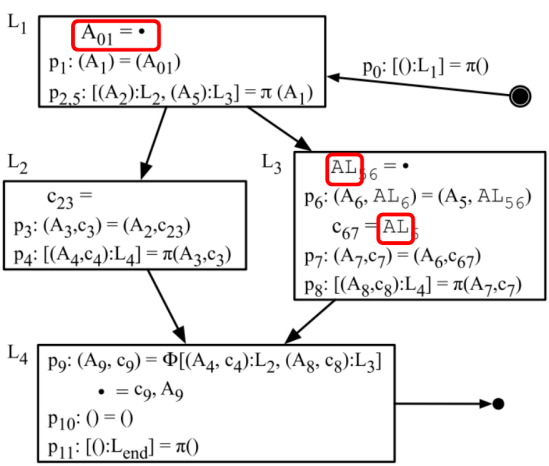
Пример решения пазла между p_x и p_{x+1}

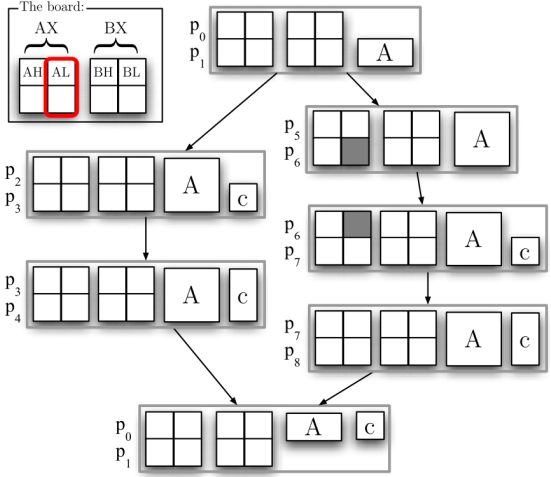
Live-in переменные: C, D, E, f

Live-out переменные: A, b, E, f

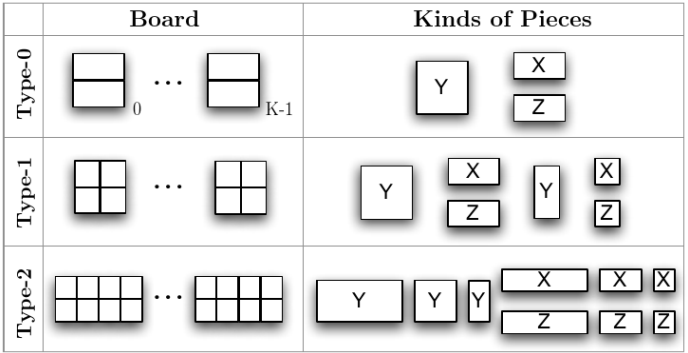


Пример. Padding



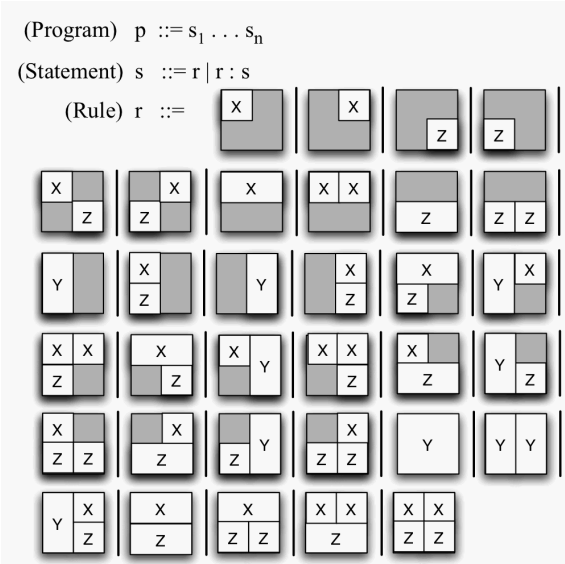


Решение пазлов типа 1

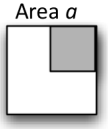


- Предлагаемый подход: заполнять по одной клетке за раз
- Для каждого квадрата:
  - Заполняем фигурками X или Z пока не заполнится вся доска
- Решить задачу

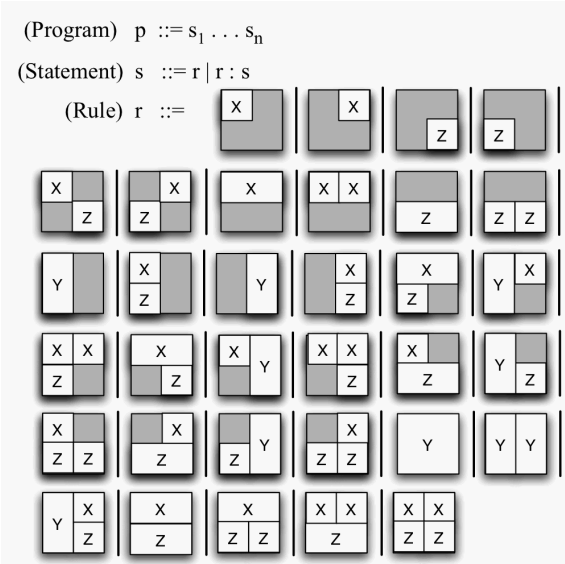
Решаем пазлы типа 1 визуальным языком (1/3)



- Правило = как заполнять зоны
- Правило состоит из
  - шаблона: что уже заполнено (подходит/неподходить под зону)
  - стратегии: какие виды фигурок класть и куда
- Правило r можно применить на зоне a iff
  - r подходит к a
  - доступны фигурки для данной стратегии

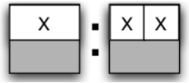


Решаем пазлы типа 1 визуальным языком (2/3)

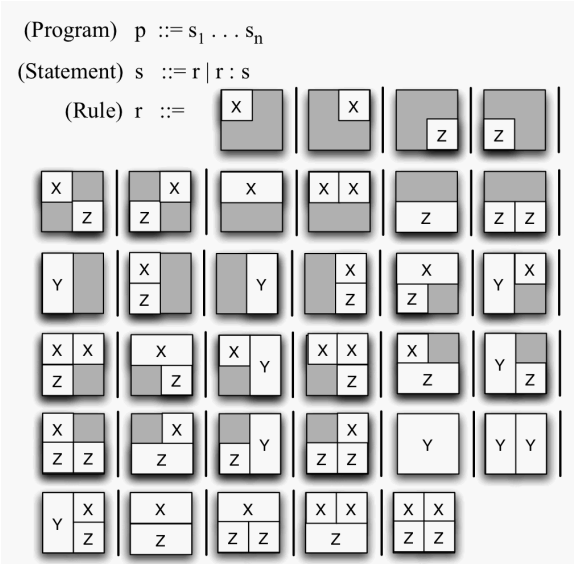


Условие успешного завершения:

- Программа успешно завершается iff все утверждения завершаются
- Правило r можно применить на зоне a iff
  - r подходить к a
  - доступны фигурки для данной стратегии
- Условие $(r : s)$ успешно завершается iff
  - r успешно завершается, или
  - s успешно завершается



Решаем пазлы типа 1 визуальным языком (3/3)

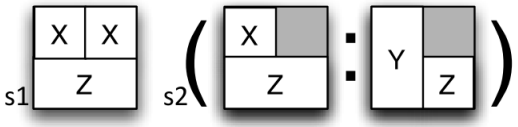


Исполнение решателя пазлов

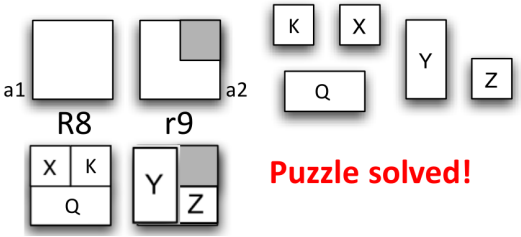
- Для каждой инструкции $s_1, \ldots, s_n$
  - Для каждой зоны a, такой что паттерн s_i подходит к a
    - Применить s_i к a
    - Если s_i закончилось и ошибкой, прервать и сообщить об ошибке

Исполнение решателя пазлов: пример

- Визуальная программа-решатель



- Пазл



Puzzle solved!

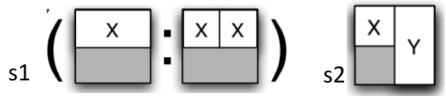
- s1 подходит только к a1
- Применение s1 к a1 завершается успешно и выдает результат



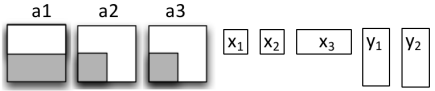
- s2 подходит только к a2
- Применяем s2 к a2
  - Применим первое правило s2: неудача
  - Применим второе правило s2: успех

Исполнение решателя пазлов: другой пример

- Визуальная программа-решатель

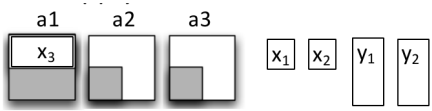


- Пазл

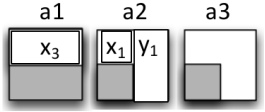


Puzzle solved!

- s1 подходит только к a1
- Применяем s1 к a1
  - Применяем первое правило s1: победа



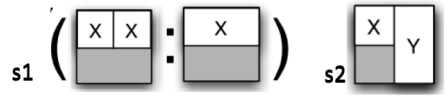
- s2 подходит и к a2, и к a3
- Применяем s2 к a2



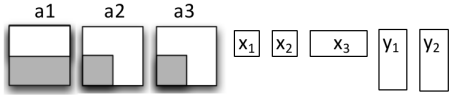
- Применяем s2 к a3

Исполнение решателя пазлов: ещё один пример

- Визуальная программа-решатель

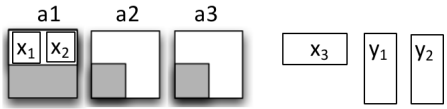


- Пазл



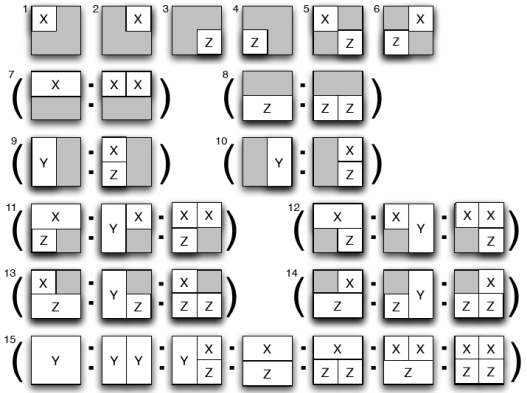
Построение правильной визуальной программы-решателя существенно!

- s1 подходит только к a1
- Применяем s1 к a1
  - Применяем второе правило s1: победа



- s2 подходит и к a2, и к a3
- Применяем s2 to a2: неудача
  Не осталось фигур типа X размера 1: мы использовали их всех

Программа-решатель пазлов типа 1



Теорема

Задача типа 1 решаема iff эта программа завершается успешно

Не решили ли мы NP задачу за полиномиальное время?

Выделение регистров ~ заполнение всех клеток.

Решена более простая задача: заполнение одной клетки за единицу времени

Решение пазлов типа 1: сложность

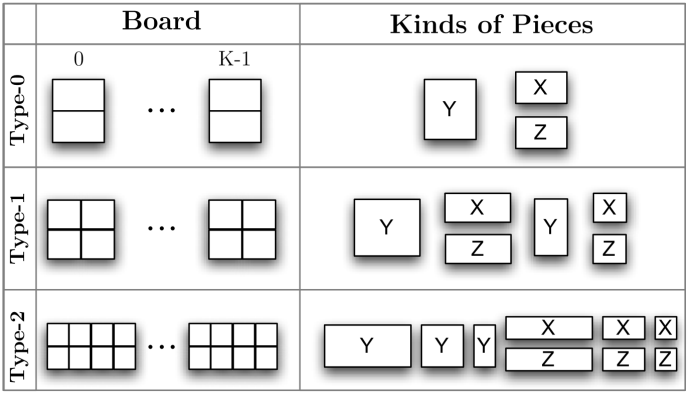
Теорема

Задача spill-free register allocation (SFRA) с предраскраской для элементарной программы P *и* K *регистров решается за* $\mathrm{O}(|\mathrm{P}| \times \mathrm{K})$ *времени.*

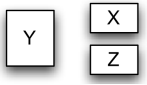
Для одной инструкции из P:

- Применение правила к зоне: $\mathrm{O}(1)$
- Применяется константное количество правил к каждой зоне
- Исполнение на поле с K зонами занимает $\mathrm{O}(\mathrm{K})$ времени

Решаем пазлы типа 0 (SFRA)



- Уложить все Y-фигурки



- Затем уложить все X и Z фигурки

Spilling в общем случае

- Если алгоритм для решения SFRA не может решить пазл (т.е. количество имеющихся регистров не достаточно), то spill
- Наблюдение: преобразование программы в элементарную форму (без φ и π) создаёт семейства переменных для каждой оригинальной переменной
- Чтобы сделать spill:
  - Выбираем переменную v в оригинальной программе
  - Spillим все переменные из элементарной формы, которые относятся к тому же семейству, что и v
  - Для пазлов это будет означать выкидывание фигурок пазла

Теорема (Сложность)

Выделение регистров в присутствии предраскрашивания (pre-coloring) и spilling семейства переменных для элементарного представления — NP-полная задача.

Ссылки & Acknowledgements I


Andrew W. Appel. *Modern Compiler Implementation in ML*. 1997.


Massimiliano Poletto и Vivek Sarkar. «Linear scan register allocation». В: *ACM Trans. Program. Lang. Syst.* 5 (1999). URL: https://doi.org/10.1145/330249.330250.


Fernando Magno Quintão Pereira и Jens Palsberg. «Register allocation by puzzle solving». В: 2008. URL: https://doi.org/10.1145/1375581.1375609.


Fernando Magno Quintão Pereira. *Linear scan register allocation (a tutorial)*. 2024. URL: http://compilers.cs.ucla.edu/fernando/projects/puzzles/tutorial.


Omri Traub, Glenn Holloway и Michael D. Smith. «Quality and speed in linear-scan register allocation». В: *SIGPLAN Not.* 5 (1998). URL: https://doi.org/10.1145/277652.277714.

Ссылки & Acknowledgements II

- Kathryn S. McKinley. *CS 380C. Register Allocation*. URL: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/17/Slides17.pdf.
- Keith Schwarz. *CS143: Compilers. Lecture 17. Register Allocation*. 2012. URL: https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/17/Slides17.pdf.